

Reg. No:

--	--	--	--	--	--	--	--	--	--



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 1 - ANALYTICAL ASSIGNMENT (SET I)

Programme : **B.E / B.Tech**

Course Code : **MLA0407**

Course Name : **Deep Learning**

Duration : **1 Hr.**

Max Marks: **50**

Answer all the Questions

1	a) Apply the Backpropagation algorithm to a simple neural network with one hidden layer and explain how the weights are updated using the gradient of the loss function. (5) b) Analyze the effect of learning rate, batch size, and number of epochs on the training performance of a deep neural network. Explain how inappropriate values can lead to slow convergence or unstable training. (5)	(10)	(CO1)	(BL3, 4)
2	a) Apply a Convolutional Neural Network (CNN) to an image-classification problem and explain the role of convolution, ReLU, pooling, and fully connected layers in extracting and classifying image features. (5) b) Analyze the causes of overfitting in CNNs and evaluate how techniques such as Dropout, Data Augmentation, Batch Normalization, and Early Stopping help improve generalization. (5)	(10)	(CO2)	(BL3, 4)
3	a) Apply a Recurrent Neural Network (RNN) to a sequence-prediction problem and explain how hidden states are used to retain information from previous time steps. (5) b) Analyze the Vanishing Gradient and Exploding Gradient problems in RNNs and explain how LSTM and GRU architectures overcome these limitations. (5)	(10)	(CO3)	(BL3, 4)
4	a) Apply Transfer Learning using a pre-trained CNN model such as VGG, ResNet, or MobileNet for a new image-classification task. Explain the steps involved in feature extraction and fine-tuning. (5) b) Evaluate the advantages and limitations of Transfer Learning compared with training a deep neural network from scratch, considering dataset size, training time, computational requirements, and accuracy. (5)	(10)	(CO4)	(BL4, 5)
5	a) Apply an Autoencoder to an image-compression or anomaly-detection problem and explain how the encoder, latent representation, and decoder work together to reconstruct the input. (5) b) Analyze the performance of CNN, RNN/LSTM, and Autoencoder architectures for different deep learning applications such as image classification, sequence prediction, and anomaly detection. Justify which architecture is most suitable for each task. (5)	(10)	(CO5)	(BL4, 5)

Question 1: Backpropagation and Hyperparameters

a) Backpropagation Algorithm in a Single-Hidden-Layer Neural Network

Backpropagation (backward propagation of errors) is the primary algorithm used to train neural networks. It computes the gradient of the loss function with respect to the network's weights, allowing the model to minimize error through gradient descent.

Consider a simple neural network with an input layer (X), one hidden layer (H), and an output layer (Y). The process occurs in two main phases:

1. The Forward Pass:

The network takes an input and calculates the predicted output.

- Hidden Layer: Computes the weighted sum of inputs and applies an activation function (like Sigmoid or ReLU).

$$Z1 = (W1 * X) + b1$$

$$A1 = \text{Activation}(Z1)$$

- Output Layer: Computes the final prediction.

$$Z2 = (W2 * A1) + b2$$

$$\text{Predicted_Y} = \text{Activation}(Z2)$$

- Loss Calculation: The difference between the prediction (Predicted_Y) and the actual target (Y) is calculated using a Loss Function (L), such as Mean Squared Error.

2. The Backward Pass (Backpropagation):

To update the weights, we must find out how much each weight contributed to the total loss. We do this by calculating the partial derivative (represented by 'd') of the loss with respect to each weight using the Chain Rule of Calculus.

- Gradients at Output Layer: We calculate how the loss changes with respect to the output weights (W_2).

$$dL/dW_2 = (dL/d\text{Predicted_Y}) * (d\text{Predicted_Y}/dZ_2) * (dZ_2/dW_2)$$

- Gradients at Hidden Layer: The error is propagated backward to the hidden layer weights (W_1).

$$dL/dW_1 = (dL/d\text{Predicted_Y}) * (d\text{Predicted_Y}/dZ_2) * (dZ_2/dA_1) * (dA_1/dZ_1) * (dZ_1/dW_1)$$

3. Weight Update:

Once the gradients are computed, the weights and biases are updated in the opposite direction of the gradient to minimize the loss.

$$\text{New_Weight} = \text{Old_Weight} - (\text{Learning_Rate} * dL/dW)$$

b) Effect of Learning Rate, Batch Size, and Epochs

The training performance of a deep neural network is highly dependent on its hyperparameters.

- Learning Rate: Determines the step size the model takes when updating weights.
 - Too High:* The model takes steps that are too large, overshooting the optimal minimum. This leads to unstable training, erratic loss fluctuations, or complete divergence.
 - Too Low:* The weight updates are minuscule. Training becomes agonizingly slow and the model is highly prone to getting stuck in suboptimal local minima.

- **Batch Size:** The number of training samples processed before updating the model's weights.
 - *Large Batch Size:* Provides a highly accurate estimate of the gradient, leading to stable but computationally heavy updates. However, it can cause the model to converge to "sharp" minima, which generalize poorly to unseen data.
 - *Small Batch Size:* Introduces noise into the gradient calculation. While it makes the training curve look erratic, this noise actually helps the model bounce out of local minima, often leading to better generalization.
- **Number of Epochs:** The number of times the entire training dataset passes through the network.
 - *Too Few:* The model hasn't had enough time to learn the underlying patterns, leading to underfitting.
 - *Too Many:* The model starts memorizing the noise and exact details of the training data rather than general patterns, leading to overfitting.

Question 2: Convolutional Neural Networks (CNNs)

a) CNNs in Image Classification

CNNs are purpose-built to process grid-like data such as images. Instead of flattening an image immediately (which destroys spatial relationships), CNNs process images in chunks.

- **Convolutional Layer:** This layer acts as a feature extractor. It slides (convolves) small matrices called filters (or kernels) across the input image. Each filter detects specific features—early layers detect simple edges or textures, while deeper layers detect complex shapes like eyes or wheels. This produces a "feature map."
- **ReLU (Rectified Linear Unit):** This is the activation function applied after convolution. It introduces non-linearity by converting all negative pixel values in the feature map to zero. This helps the network learn complex, non-linear mappings and mitigates the vanishing gradient problem.
- **Pooling Layer (Downsampling):** Pooling (usually Max Pooling) slides a window over the feature map and keeps only the maximum value in that window. This drastically reduces the computational load, reduces the image dimensions, and provides spatial invariance (meaning the network can recognize a feature even if it's slightly shifted in the image).
- **Fully Connected (FC) Layers:** At the end of the network, the 2D feature maps are flattened into a 1D vector. The FC layers look at these high-level extracted features and output a final probability distribution (e.g., using Softmax) to classify the image into categories like "cat" or "dog."

b) Overfitting in CNNs and Mitigation Techniques

Overfitting occurs when a CNN is too complex relative to the amount of training data. The model essentially memorizes the training images, resulting in near-perfect training accuracy but terrible validation/test accuracy.

- **Dropout:** During training, Dropout randomly "turns off" a percentage of neurons in a layer. This forces the network not to rely on any single neuron or specific feature combination, promoting redundant, robust feature learning.
- **Data Augmentation:** Small datasets cause overfitting. Augmentation artificially expands the dataset by applying random transformations to existing images—such as rotations, flips, zooming, and color shifting. This forces the model to learn invariant features rather than memorizing exact pixel layouts.
- **Batch Normalization:** This technique normalizes the outputs of a previous layer before passing them to the next. While primarily used to speed up training and stabilize gradients, it adds a slight noise to the network, which acts as a mild regularizer and helps prevent overfitting.
- **Early Stopping:** This monitors the model's performance on a validation set during training. If the training loss continues to decrease but the validation loss starts to increase (the classic sign of overfitting), Early Stopping halts the training process and restores the best-performing weights.

Question 3: Recurrent Neural Networks (RNNs)

a) RNNs for Sequence Prediction

Unlike standard feedforward networks that process independent inputs, RNNs are designed for sequential data (like time series, text, or audio). They do this by maintaining an internal memory, or hidden state.

At any given time step (t), the RNN receives two inputs:

1. The current data point in the sequence: X_t
2. The hidden state from the previous time step: $H_{(t-1)}$

The network calculates the new hidden state using both inputs:

$$H_t = \text{Tanh}((W_{xh} * X_t) + (W_{hh} * H_{(t-1)}) + b)$$

This means the output at time (t) is influenced not just by the current word or data point, but by the entire history of the sequence encoded in the previous hidden state. If predicting the next word in the sentence "The clouds are in the...", the hidden state retains the context of "clouds" to help accurately predict "sky".

b) Vanishing/Exploding Gradients and LSTM/GRU

Standard RNNs struggle with long sequences due to how gradients are calculated through time. During backpropagation, gradients are multiplied by the recurrent weight matrix repeatedly for every time step.

- Exploding Gradients: If the weights are slightly greater than 1, repeated multiplication causes the gradients to grow exponentially, causing the model to crash.
- Vanishing Gradients: If the weights are less than 1, repeated multiplication causes the

gradients to shrink to zero. The network stops learning because it cannot update weights based on distant past information (it "forgets" long-term context).

How LSTMs and GRUs fix this:

- LSTM (Long Short-Term Memory): Replaces the simple RNN hidden layer with a complex "cell". It introduces a Cell State, which acts like a conveyor belt carrying long-term memory straight down the sequence with minimal linear interactions (preventing gradients from vanishing). It uses three "gates" (Forget, Input, Output) made of sigmoid layers to meticulously control what information is kept, added, or discarded from the cell state.
- GRU (Gated Recurrent Unit): A streamlined version of the LSTM. It combines the forget and input gates into a single "Update Gate" and merges the cell state and hidden state. It is computationally faster than an LSTM while achieving similar performance in solving the vanishing gradient problem.

Question 4: Transfer Learning

a) Applying Transfer Learning with Pre-trained CNNs

Transfer learning takes a model trained on a massive dataset (like ImageNet, which has millions of images and 1,000 classes) and repurposes it for a new, often smaller, task (like classifying X-rays). Models like VGG16, ResNet, or MobileNet have already learned exceptional feature extractors (edges, shapes, textures) in their convolutional layers.

Steps Involved:

1. Feature Extraction:

- Load the pre-trained model but remove its final Fully Connected classification head (since it's configured for 1,000 ImageNet classes).
- "Freeze" the weights of the convolutional base so they do not update during training.
- Add a new custom Fully Connected layer on top, matching the number of classes in your new dataset.
- Train the network. Only the weights of your newly added classification head will be updated. The frozen base simply acts as a feature extractor.

2. Fine-Tuning (Optional but common):

- After the new classifier is trained, you "unfreeze" the top few convolutional layers of the pre-trained base.
- Retrain the entire model using a very small learning rate. This gently adjusts the highly abstract features in the deeper layers to be more specific to your specific dataset, without wrecking the fundamental edge-detection weights in the early layers.

b) Transfer Learning vs. Training from Scratch

- **Dataset Size:** Training from scratch requires massive amounts of labeled data (tens of thousands of images) to learn basic features without overfitting. Transfer learning requires very little data because the model already knows how to "see" features; it just needs to map them to new labels.
- **Training Time:** Training from scratch can take days or weeks depending on network depth. Transfer learning is extremely fast (often minutes or hours) because you are mostly just training a single dense layer.
- **Computational Requirements:** Training from scratch is very high and requires expensive GPU clusters to optimize millions of initialized weights. Transfer learning is low to moderate.
- **Accuracy:** Training from scratch is prone to poor accuracy and high overfitting if the dataset isn't large enough. Transfer learning usually yields significantly higher accuracy, as it leverages the robust generalization learned from millions of prior images.

Question 5: Autoencoders and Architecture Selection

a) Autoencoders for Anomaly Detection/Compression

An Autoencoder is an unsupervised neural network designed to learn a compressed representation of data. It forces the input through a narrow bottleneck and attempts to reconstruct the original input on the other side.

- The Encoder: Takes the high-dimensional input data (X) and compresses it through successively smaller dense layers.
- The Latent Representation (Bottleneck): The smallest layer in the middle of the network. It holds the compressed, core features of the input data (Z).
- The Decoder: Takes the compressed latent vector (Z) and attempts to reconstruct the original input, outputting a reconstructed version (X_{prime}).

How it works for Anomaly Detection:

The model is trained *only on normal, healthy data*. The loss function (Reconstruction Error, which is the mathematical difference between X and X_{prime}) forces the network to become very good at compressing and reconstructing normal data.

When an anomaly (e.g., a defective manufacturing part or fraudulent transaction) is fed into the network, the encoder doesn't know how to compress these unfamiliar features, and the decoder fails to reconstruct it accurately. Therefore, a high reconstruction error serves as an anomaly trigger.

b) Comparing CNN, RNN/LSTM, and Autoencoder Architectures

Different deep learning architectures are mathematically optimized for specific data structures.

1. CNNs (Best for Spatial Data - Image Classification):

- *Justification:* CNNs excel at grid topologies. Because pixels close to each other form logical features (edges, textures), CNNs use localized filters and shared weights to capture these spatial hierarchies. An RNN would fail here because it expects sequential order, and an Autoencoder is generally not robust enough for complex supervised classification without massive modifications.

2. RNN/LSTM (Best for Temporal/Sequential Data - Sequence Prediction):

- *Justification:* Time series forecasting, text processing, and speech recognition require an understanding of sequence and context. A word's meaning depends on the word before it. LSTMs maintain this state across time. CNNs lack memory of sequence order, making them generally unsuitable for tasks where temporal history dictates the future.

3. Autoencoders (Best for Unlabeled Data - Anomaly Detection / Dimensionality Reduction):

- *Justification:* When you lack labeled data ("defective" vs "non-defective"), supervised models like CNNs and RNNs cannot be trained effectively. Autoencoders bypass this by using the input itself as the label. They are the ideal choice for finding outliers in massive, unstructured datasets or for compressing high-dimensional data.

Assessment Rubrics

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing
Conceptual Understanding of AI Techniques	10%	Demonstrates a comprehensive and accurate understanding of the underlying AI concept (agents, search, logic, or learning)	Demonstrates good understanding with minor gaps	Basic understanding; some concepts unclear or misapplied
Correct Application of Algorithms/Techniques	15%	Correctly and completely applies the specified algorithm/technique (PEAS, A*, Resolution, Decision Tree, etc.) with accurate steps	Applies the algorithm/technique correctly with minor errors in steps	Partially correct application with noticeable errors
Analytical Reasoning & Justification	10%	Provides thorough, logical reasoning with well-justified classifications/derivations	Provides logical reasoning with minor gaps in justification	Basic reasoning with limited or weak justification
Critical Evaluation & Comparison	10%	Insightful, well-balanced evaluation/comparison with clear engineering trade-offs	Good evaluation/comparison with reasonable trade-off discussion	Basic evaluation/comparison with limited depth
Presentation & Engineering Documentation Standards	5%	Clear, well-structured, professionally presented with correct notation/diagrams	Good structure and presentation with minor lapses	Basic presentation with some structural/notational issues

Code of Conduct

I T Bhupal/192425243 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic-integrity rules will result in disciplinary action.

Signature of the Student: bhupalreddy.

_____ Faculty In-charge Name: Date:	_____ Course Coordinator Name: Date:	_____ Head of Department Name: Date:
---	--	--